

FORMATION COMPLÈTE · 4 JOURS · NIVEAU DÉBUTANT

Tester un logiciel avec l'IA générative

Acquérir le bon état d'esprit, concevoir des tests, automatiser, puis contextualiser les modèles pour votre métier — chaque notion expliquée et illustrée par l'exemple.

 ChatGPT · Claude · Mistral

 Fonctionnel & technique

 Pack de 102 prompts en complément

 Digital House Company

hello@dhcompany.pro

VOTRE FORMATEUR



Joël Parfait Kuate

Ingénieur IA · Consultant & Formateur certifié

- ▷ **Master** en Architecture Logicielle & Big Data
- ▷ **7+ ans** de formation — **600+ organisations** formées en Belgique
- ▷ Fondateur de **Digital House Company** & MS JOYCE SARL
- ▷ Auteur de « **L'Afrique et l'Intelligence Artificielle** » (2024)
- ▷ Projets : ia4africa.org · FormIA · SANDYKIT · Jurisis.be · Indibot.be

Ma méthode : tout expliquer simplement, tout montrer par l'exemple, et vous rendre autonome. hello@dhcompany.pro

PROGRAMME

4 journées qui se construisent l'une sur l'autre



Jour 1

Acquérir le bon état d'esprit avec l'IA générative.



Jour 2

Techniques de tests et ingénierie des prompts.



Jour 3

tech

Techniques avancées pour développeurs et automaticiens.



Jour 4

Contextualiser les modèles d'IA pour des contextes métier.

Chaque journée se termine par des **travaux pratiques** sur ChatGPT, Claude ou Mistral.

MODE D'EMPLOI

Comment lire ce support



Le concept

Chaque notion est d'abord expliquée simplement, sans présupposé.



L'exemple

Chaque notion est ensuite illustrée par un cas concret encadré en jaune.



Le pack

Les encadrés verts renvoient aux prompts à utiliser depuis le pack.

Les blocs marqués **tech** sont des approfondissements techniques. Un profil fonctionnel peut les survoler.

VOTRE BOÎTE À OUTILS

Un pack de prompts vous accompagne

- ◆ Un **pack de 102 prompts** (codes FN-01 à FN-102) couvre toutes les facettes du test.
- ◆ Un pack de **55 patterns de context engineering** (CE-01 à CE-55) cadre l'IA sur vos données.
- ◆ Chaque atelier renvoie à des prompts précis : vous n'inventez rien de zéro.
- ◆ Vous apprenez à **choisir et remplir** le bon prompt, pas à les mémoriser.

Complément : gardez le guide du pack ouvert à côté de ce support pendant les exercices.

JOUR 1 · CHAPITRE 1

Acquérir le bon état d'esprit avec l'IA générationnelle

Comprendre l'outil en profondeur avant de l'utiliser



D'abord, qu'est-ce qu'un LLM ?

LLM veut dire « Large Language Model », ou grand modèle de langage. C'est un programme entraîné sur d'énormes quantités de texte, capable de produire du langage : répondre, rédiger, expliquer, générer du code.

ChatGPT, Claude et Mistral sont des LLM. Ce sont les trois outils que nous utiliserons en travaux pratiques.

Comment « réfléchir » un LLM ?

- ◆ Il ne comprend pas comme un humain. Il **prédit le mot suivant**.
- ◆ À chaque mot, il calcule lequel est le plus probable selon le contexte.
- ◆ Il enchaîne ces prédictions pour former une phrase cohérente.
- ◆ C'est puissant pour produire du texte, mais cela explique aussi ses erreurs.

Exemple : après « Le ciel est... », le modèle calcule que « bleu » est très probable. Il l'écrit. Il ne « sait » pas que le ciel est bleu : il calcule une probabilité.

JOUR 1 · L'INTÉRÊT POUR LE TEST

Améliorer les activités de test avec les LLM

- ◆ Le LLM accélère toutes les tâches **réactionnelles et répétitives** du testeur.
- ◆ Rédiger des cas de test, des données, des rapports : beaucoup plus vite.
- ◆ Il propose des idées et des cas auxquels on n'aurait pas pensé.
- ◆ Le testeur garde le **jugement** ; l'IA fournit la **matière première**.

Avant / après sur une vraie tâche



Sans IA

Le testeur rédige à la main 15 cas de test pour un formulaire. Temps : environ 1 heure. Risque d'oublier des cas limites.



Avec IA

Le testeur décrit le formulaire, l'IA propose 20 cas en 30 secondes. Le testeur trie et complète en 15 minutes.

Gain : le temps passe de 60 à 15 minutes, et la couverture est souvent meilleure. Le travail de tri reste humain.

Le risque de sécurité

- ◆ Tout ce que vous tapez dans un outil grand public peut être **conservé**.
- ◆ Coller des données clients ou du code confidentiel = risque de fuite.
- ◆ Règle simple : **aucune donnée sensible** dans un outil non validé par l'entreprise.
- ◆ Préférer les versions entreprise (cloud privé) pour les données réelles.

Exemple à éviter : coller la base clients réelle pour « générer des tests ». À la place, on utilise des données factices (voir Jour 2).

Le risque législatif

- ◆ **RGPD** : interdiction de traiter des données personnelles sans cadre.
- ◆ **AI Act** européen : encadre les usages de l'IA selon leur risque.
- ◆ **Propriété intellectuelle** : attention au code ou texte généré et à son origine.
- ◆ Le testeur doit savoir ce qu'il a le droit de soumettre à l'IA.

Exemple : demander à l'IA d'analyser un contrat client nominatif viole le RGPD. Anonymiser d'abord, toujours.

Le risque énergétique

- ◆ Chaque requête à un LLM consomme de l'électricité dans un centre de données.
- ◆ Multiplié par des millions d'utilisateurs, l'impact devient significatif.
- ◆ Bonne pratique : **un prompt bien pensé** évite dix essais inutiles.
- ◆ Utiliser l'IA quand elle apporte vraiment de la valeur, pas par réflexe.

Un usage sobre et réfléchi est à la fois plus écologique et plus efficace.

Erreurs et hallucinations : c'est quoi ?

Une **hallucination** est une réponse fausse que l'IA présente avec assurance. Le modèle ne ment pas volontairement : il calcule une suite de mots probable, même quand il ne « sait » pas.

C'est la conséquence directe du mécanisme de prédiction vu plus tôt. L'IA préfère répondre quelque chose plutôt que rien.

Repérer une hallucination

- ◆ L'IA cite une « fonction **validateIBAN()** » qui n'existe pas dans votre code.
- ◆ Elle invente un article de loi au numéro précis mais inexistant.
- ◆ Elle affirme un chiffre statistique sans source vérifiable.

Parade : toujours vérifier les faits, les noms et les chiffres. Demander à l'IA d'indiquer ses sources. Ne jamais livrer une sortie non relue.

L'ingénierie des prompts : la recette

- ◆ **Rôle** — dire à l'IA qui elle doit être (« un testeur senior »).
- ◆ **Tâche** — ce qu'elle doit faire (« génère des cas de test »).
- ◆ **Contexte** — vos règles, vos contraintes, votre situation.
- ◆ **Format** — la forme attendue (« un tableau »).
- ◆ **Exemples** — un ou deux modèles pour cadrer le style.

Moyen mnémotechnique : **R-T-C-F-E**. Si une réponse déçoit, un ingrédient manque presque toujours.

Un mauvais prompt vs un bon prompt

```
# Mauvais prompt – vague, sans contexte ni format  
"Écris des tests pour le login."
```

```
# Bon prompt – R-T-C-F-E
```

```
Rôle : Tu es un testeur fonctionnel rigoureux.
```

```
Tâche : Génère les cas de test de la connexion.
```

```
Contexte : email + mot de passe 8-20 caractères, blocage après 3 échecs.
```

```
Format : tableau ID | étapes | résultat attendu.
```

```
Couvre : cas passants, erreurs, valeurs limites.
```

Pack : CE-01 (rôle expert), FN-90 (expliquer un concept simplement).

Prompt vs context engineering



Prompt engineering

Bien formuler la demande : rôle, tâche, format. Agit sur une seule question.



Context engineering

Fournir le bon contexte : spécifications, règles, exemples. L'IA travaille sur VOS données.

Exemple : coller la règle « préavis de congé : 2 jours ouvrés » fait que l'IA teste exactement cette règle, au lieu d'en inventer une.

IA, automatisation et test des logiciels

TYPE DE TEST	CE QUE L'IA APPORTE
Manuel	Génère les cas et les données à exécuter
Exploratoire	Propose des chartes et des idées d'attaque
Automatisé	Génère le code des scripts de test
Rapport	Résume et interprète les résultats

L'IA n'automatise pas à votre place : elle vous aide à **préparer** plus vite et mieux.

Réaliser ses premiers prompts

45 min · binôme · ChatGPT / Claude / Mistral

- 1 Posez un prompt vague à l'IA et observez la réponse.
- 2 Reprenez le même besoin en appliquant la recette R-T-C-F-E.
- 3 Comparez les deux réponses et notez ce qui a changé.
- 4 Provoquez une hallucination volontairement, puis apprenez à la détecter.

Livable : un prompt « avant / après » et un exemple d'hallucination repérée.

Pack à utiliser : CE-01 (rôle), CE-26 (interdire l'invention), FN-90.

JOUR 2 · CHAPITRE 2

Techniques de tests et ingénierie des prompts

Produire de vrais artefacts de test avec l'IA

J2

JOUR 2 · CONCEVOIR (1/2)

Élaborer des tests avec l'assistance de l'IA

1

Exigence

On part d'une user story ou d'une règle.

2

Prompt

On demande des cas de test structurés.

3

Brouillon IA

L'IA propose un tableau de cas.

4

Relecture

On corrige, complète, supprime les doublons.

L'étape 4 est non négociable. L'IA propose, le testeur décide.

D'une user story à des cas de test

User story

En tant qu'employé, je veux poser un congé.

Cas générés par l'IA (extrait)

TC-01 : solde suffisant, préavis ok → **accepté**

TC-02 : solde insuffisant → **refusé**

TC-03 : congé demandé pour demain → **refusé (préavis)**

TC-04 : chevauchement d'un congé existant → **refusé**

Pack : FN-15 (cas depuis user story), FN-16 (cas depuis critères).

Les partitions d'équivalence

Idée : **regrouper les valeurs qui se comportent pareil** et n'en tester qu'une par groupe. Inutile de tester 1 000 valeurs si elles donnent toutes le même résultat.

Exemple — champ âge (18 à 99) : classe invalide basse → tester 10. Classe valide → tester 40. Classe invalide haute → tester 150. Trois tests suffisent.

L'analyse des valeurs limites

Les bugs adorent se cacher **aux frontières**. On teste donc juste avant, sur, et juste après chaque borne.

Exemple — mot de passe (8 à 20 caractères) : tester 7, 8, 9 (borne basse) et 19, 20, 21 (borne haute). L'erreur de décalage d'une unité est l'une des plus fréquentes.

Trier les cas générés par l'IA

- ◆ **Doublons** : deux cas qui testent exactement la même chose.
- ◆ **Cas triviaux** : sans valeur, on les supprime.
- ◆ **Hors sujet** : l'IA s'est égarée, on écarte.
- ◆ **Oublis** : on complète à la main ce qui manque.

Pack : FN-19 (réduire les doublons), FN-95 (vérifier l'exhaustivité).

JOUR 2 · EXPLORATOIRE (1/2)

Assister le test exploratoire

Le test exploratoire cherche les bugs que les scripts ne voient pas. On explore librement, en se demandant « **et si je faisais ça ?** ». L'IA enrichit cette exploration.

Le test scripté vérifie ce qu'on sait. Le test exploratoire découvre ce qu'on ignorait.

Des idées d'attaque générées

Le vide

Champ vide, panier vide, liste vide.

Le trop

Texte de 10 000 caractères, 999 articles.

Le spécial

Émojis, accents, balises HTML dans un champ.

Pack : FN-31 (charte de session), FN-32 (heuristiques d'attaque).

Créer rapidement des jeux de données

- ◆ L'IA génère des données **valides, invalides et limites** en quelques secondes.
- ◆ Elle produit des données **réalistes mais factices** : noms, emails, IBAN de test.
- ◆ Des données concrètes rendent les cas immédiatement exécutables.
- ◆ Règle absolue : **jamais de données réelles** en environnement de test.

Un jeu de données généré

CHAMP	VALIDE	INVALIDE	LIMITE
Email	a@b.fr	a@b	très long
Âge	40	-5	18 et 99
Téléphone	+32470123456	12	avec espaces

Pack : FN-27 (valides/invalides), FN-28 (limites), FN-30 (réalistes), FN-29 (anonymiser).

Analyser un rapport de tests

- ◆ L'IA transforme des **résultats bruts** en rapport lisible.
- ◆ Elle propose des **KPI** : couverture, avancement, taux de réussite.
- ◆ Elle traduit les chiffres pour la direction, sans jargon.
- ◆ Vous validez l'interprétation avant de la diffuser.

Exemple : « 120 cas, 90 exécutés, 80 réussis, 3 bugs majeurs » devient une synthèse claire avec une recommandation de livraison.

Visualiser les résultats

- ◆ Un tableau de bord vaut mieux qu'une longue liste de chiffres.
- ◆ L'IA suggère le bon graphique : barres pour comparer, courbe pour suivre.
- ◆ Elle met en avant ce qui doit alerter : bugs critiques, zones non couvertes.
- ◆ Objectif : une décision claire en un coup d'œil.

Pack : FN-71 (rapport d'avancement), FN-72 (synthèse direction), FN-73 (KPI), FN-74 (interpréter).

Créer des artefacts de test assistés

60 min · binôme · comptes IA

- 1 Choisissez une user story simple (ex. réinitialiser un mot de passe).
- 2 Générez les cas de test, puis triez-les à la main.
- 3 Générez un jeu de données factices et une charte exploratoire.
- 4 Faites résumer un faux rapport de test en synthèse pour la direction.

Livrable : un mini-dossier de test complet (cas, données, charte, synthèse).

Pack à utiliser : FN-15, FN-21, FN-27, FN-31, FN-72.

JOUR 3 · CHAPITRE 3 · TECHNIQUE

Techniques avancées pour développeurs et automaticiens

Du test unitaire à l'agent IA



JOUR 3 · UNITAIRE (1/2)

Qu'est-ce qu'un test unitaire ? tech

Un **test unitaire** vérifie une petite brique de code isolée — une seule fonction. Si la fonction fait une erreur, le test échoue immédiatement. C'est le filet de sécurité du développeur.

Pour les profils fonctionnels : retenez le principe. Pas besoin de savoir l'écrire pour comprendre sa valeur.

Un test unitaire généré tech

```
// Fonction : remise (10% dès 100€, 20% dès 500€)
def test_remise():
    assert remise(99) == 0 # juste sous le seuil
    assert remise(100) == 10 # pile au seuil
    assert remise(500) == 100 # second palier
    # L'IA a bien placé les tests aux frontières
```

Pack : FN-80 (cas automatisables), FN-22 (valeurs limites appliquées au code).

JOUR 3 · E2E (1/2)

Les tests de bout en bout (E2E) tech

Un test **End-to-End** simule un parcours utilisateur complet, du début à la fin : ouvrir la page, remplir un formulaire, cliquer, vérifier le résultat — comme un vrai utilisateur, en automatique.

L'IA peut générer ces scripts à partir d'un scénario décrit en français.

Un test Playwright généré tech

```
// Scénario : "teste une connexion valide"
test('connexion valide', async ({ page }) => {
  await page.goto('/login');
  await page.fill('#email', 'julie@mail.com');
  await page.fill('#password', 'MotDePasse123');
  await page.click('text=Se connecter');
  await expect(page).toHaveURL(/dashboard/);
});
```

Exemple : ce script a été généré à partir d'une seule phrase en français. Le testeur le relit puis l'exécute.

JUnit, WebDriver, Playwright, Cypress

tech

OUTIL	À QUOI IL SERT
JUnit	Tests unitaires en Java
Selenium WebDriver	Automatisation navigateur, historique
Playwright	Automatisation navigateur moderne et rapide
Cypress	Tests E2E orientés front-end web

L'IA peut générer du code pour chacun de ces outils à partir d'un scénario décrit en clair.

Insérer le code généré dans l'existant tech

- ◆ L'IA produit un test, mais il faut le **brancher** au projet réel.
- ◆ Vérifier les imports, les noms de variables, les sélecteurs de page.
- ◆ Lancer le test : s'il échoue, comprendre pourquoi avant de corriger.
- ◆ Ne jamais copier-coller sans relire : l'IA peut se tromper de contexte.

Exemple : l'IA utilise `#email` alors que votre page utilise `#user-email`. Le test échoue tant que vous n'avez pas corrigé le sélecteur.

JOUR 3 · AGENT (1/2)

L'agent IA comme assistant de test tech

Un **agent IA** ne se contente pas de répondre : il enchaîne des actions. Il peut lire le code, lancer un test, observer le résultat et rapporter — en suivant un objectif que vous lui donnez.

L'agent décuple la productivité, mais il reste un copilote : il ne décide pas seul de livrer.

Comment fonctionne un agent de test tech

1

Objectif

« Vérifie le parcours d'achat. »

2

Action

Il navigue et exécute les étapes.

3

Observation

Il lit les résultats et capture les preuves.

4

Rapport

Il signale ce qui a échoué, à confirmer.

Ses observations sont des **pistes à vérifier**, jamais des verdicts définitifs.

Générer du code de test et le brancher tech

60 min · binôme · VS Code + comptes IA

- 1 Faites générer des tests unitaires (JUnit) pour une fonction donnée.
- 2 Faites générer un test IHM automatisé (Playwright ou Cypress).
- 3 Insérez le code dans un projet existant (WebDriver) et exécutez-le.
- 4 Analysez le fonctionnement d'un agent IA de test en Java ou Python.

Livable : un test unitaire et un test E2E générés, exécutés et relus.

Pack à utiliser : FN-80, FN-81 (Gherkin pour automatisation), FN-46 (tests d'API).

JOUR 4 · CHAPITRE 4

Contextualiser les modèles d'IA pour des contextes métier

Faire travailler l'IA sur VOS données



Pourquoi personnaliser un modèle ?

Un modèle générique connaît le monde, mais pas **votre application**. Il ignore vos règles métier, votre vocabulaire, vos documents internes. Le personnaliser, c'est lui donner ce contexte pour des réponses sur mesure.

Sans personnalisation, l'IA reste utile mais générale. Avec, elle devient pertinente pour votre projet précis.

Trois façons de personnaliser



Prompt + contexte

On fournit les infos directement dans la demande. Simple et immédiat.



RAG

On connecte l'IA à vos documents. Elle y puise pour répondre.



Fine-tuning

On ré-entraîne le modèle sur vos exemples. Lourd, spécialisé.

On essaie toujours dans cet ordre : prompt d'abord, puis RAG, et le fine-tuning en dernier recours.

Personnaliser par le contexte

- ◆ La façon la plus simple : **donner le contexte dans le prompt.**
- ◆ Vos règles métier, votre glossaire, un exemple type.
- ◆ Aucun outil technique : ça marche dès aujourd'hui.
- ◆ C'est tout l'objet du pack de context engineering (CE-01 à CE-55).

Pack : CE-06 (fournir la spec), CE-11 (règles métier), CE-39 (glossaire).

Comprendre le RAG

RAG signifie « Retrieval Augmented Generation » : génération augmentée par la recherche. L'idée : avant de répondre, le système va **chercher vos documents** et les donne au modèle, qui répond à partir d'eux.

Résultat : l'IA répond sur la base de VOS spécifications, pas de sa mémoire générale. Beaucoup moins d'hallucinations.

Le RAG étape par étape

1

Question

Le testeur pose sa question.

2

Recherche

Le système retrouve les bons documents internes.

3

Contexte

Ces documents sont fournis au modèle.

4

Réponse

L'IA répond à partir de vos documents.

Exemple : « Quelle est la règle de préavis ? » → le RAG retrouve votre spécification RH → l'IA répond « 2 jours ouvrés » en citant le document.

Comprendre le fine-tuning

Le **fine-tuning** ré-entraîne le modèle sur vos propres exemples. On ne lui donne plus le contexte à chaque fois : on change son comportement **en profondeur**, une fois pour toutes.

Plus puissant pour des besoins très spécifiques, mais plus lourd et plus coûteux que le RAG.

Quand utiliser le fine-tuning ?

- ◆ Quand vous avez un **besoin répétitif et très spécifique**.
- ◆ Quand vous disposez de **nombreux exemples** de qualité.
- ◆ Quand le prompt et le RAG ne suffisent plus.
- ◆ Rarement nécessaire pour débiter : c'est un recours avancé.

Exemple : une équipe qui rédige des milliers de cas de test dans un format maison très précis peut fine-tuner un modèle pour qu'il l'applique d'office.

RAG ou fine-tuning ?

RAG	FINE-TUNING
Donne des connaissances à jour	Change le comportement du modèle
Rapide à mettre en place	Long et coûteux
Documents modifiables à tout moment	Réentraînement nécessaire pour changer
Idéal pour débiter	Pour besoins spécialisés et stables

Pour 90% des besoins d'un testeur, le **prompt + RAG** suffit largement.

Les deux techniques en direct

- ◆ Démonstration du **RAG** : l'IA répond à partir d'un document interne fourni.
- ◆ Démonstration du **fine-tuning** : un modèle adapté à un format précis.
- ◆ Comparaison des deux résultats sur le poste du formateur.
- ◆ Discussion : laquelle pour quel besoin dans votre équipe ?

Démonstration & opportunités futures

45 min · collectif

- 1 Assistez à la démonstration RAG et fine-tuning du formateur.
- 2 Identifiez un cas d'usage RAG dans votre propre contexte de test.
- 3 Listez vos documents internes mobilisables (specs, normes, glossaire).
- 4 Construisez votre plan d'action pour les 30 prochains jours.

Livable : une idée de RAG appliquée à votre équipe et un plan d'action à 30 jours.

Pack à utiliser : CE-06, CE-11, CE-39 (le bloc context engineering complet).

COMPLÉMENT

Le pack de prompts, votre boîte à outils à vie

À garder et réutiliser après la formation



Ce que contient le pack



102 prompts (FN)

Pour chaque facette : exigences, cas de test, données, bug, régression, API, reporting, RGPD... Chacun avec un exemple concret.



55 patterns (CE)

Pour cadrer l'IA : rôle, spécifications, règles métier, format de sortie, garde-fous, glossaire... Chacun avec un exemple.

Complément : le guide complet est fourni en Word et en texte. Gardez-le ouvert à chaque session de test.

COMPLÉMENT · EXEMPLE

Un prompt du pack, rempli

```
# FN-21 – Partitions d'équivalence
```

```
Modèle : Applique la technique des partitions d'équivalence  
au champ « {champ} » dont la règle est « {regle} ».
```

```
Rempli : ...au champ « mot de passe »  
dont la règle est « entre 8 et 20 caractères ».
```

Le principe est toujours le même : ouvrir le bon prompt, remplir les variables entre accolades, relire la réponse.

Combiner contexte et prompt

La vraie puissance vient de la **combinaison** : on empile quelques patterns de contexte, puis on ajoute le prompt. La réponse est alors fiable et adaptée à votre projet.

Exemple : CE-01 (rôle expert) + CE-26 (ne pas inventer) + CE-11 (vos règles) + FN-15 (générer les cas de test) = un message complet et précis.

5 idées à emporter

- 1 Un LLM **prédit le mot suivant** : puissant, mais faillible. On vérifie toujours.
- 2 La compétence clé est le **prompt + le contexte** : bien demander, bien cadrer.
- 3 De la conception au rapport, l'IA produit des **artéfacts de test** à relire et trier.
- 4 Côté technique, l'IA génère **tests unitaires et E2E** et pilote des agents.
- 5 Le **RAG** fait travailler l'IA sur vos données ; le pack de prompts reste votre boîte à outils.

Merci.

Des questions, un besoin de formation pour vos équipes ? Contactez-moi directement.

hello@dhcompany.pro

dhcompany.pro

ecoledunumerique.be

ia4africa.org

@DigitalMaker237 sur LinkedIn & X — Joël Parfait Kuate, Digital House Company